Chapter 10: Debugging and Optimization

Debugging and optimization are crucial skills for any programmer, allowing them to write efficient, error-free, and high-performance code. In this chapter, we will explore common debugging techniques, dive into popular debugging tools like gdb, and provide best practices for optimizing your code to make it both effective and efficient.

Common Debugging Techniques

Debugging is the process of identifying and fixing errors, or "bugs," in a program. Bugs can arise from various issues, such as incorrect logic, data inconsistencies, or unexpected behavior due to external factors. Here are some common debugging techniques used to locate and fix issues in your code:

Print Statements and Logging One of the simplest and most common ways to debug is by inserting print statements into your code. By printing the values of variables, the flow of control, or intermediate results at various points, you can track the state of your program. For example:

```
def add_numbers(a, b):
    print(f"Adding {a} and {b}")  # Debug statement
    return a + b
```

However, relying on print statements for complex programs may clutter the code and make it difficult to isolate issues. This is where logging can help. Logging allows you to record messages at various levels (e.g.,